

增強課程 2：Classic User-Exit 實戰——批號自動給號 + Number Range Object

Lecture

這題的起點是一個真實需求：批號要自動依「日期 + 流水號」格式給號，不要用系統預設的批號。這是很典型會用到 Enhancement Framework 的場景——SAP 標準的批號自動給號邏輯是固定的，要改成自訂格式，就得找到官方留的插入點，而不是去改標準程式碼本身。

先講一個重要的系統事實：批號欄位 (MCHA-CHARG, Data Element CHARG_D, Domain CHARG) 長度限制是 CHAR(10)——這是本題查證出來的第一個硬限制，直接決定了格式怎麼設計。原本設想的「日期 + 時間 + 流水號 5 碼」(YYYYMMDD8碼 + HHMMSS6碼 + 5碼 = 19碼) 完全塞不進 10 碼，這門課實際採用的格式是YYMMDD (6 碼) + 流水號 (4碼) = 10碼。

找真正的機制：三代擴充技術疊在同一個地方

用 sap-adt MCP 追查 SAP 標準批號自動給號邏輯 (Function Module VB_NEXT_BATCH_NUMBER, Function Group V01Z, 套件 VB) 的原始碼，發現一個很有教學價值的現象：同一段給號邏輯裡，三代擴充技術依序都會被呼叫，不是互斥的：

1. 新式 BAdI (GET BADI g_badi_batch_number_int / CALL BADI ...->init / ...->next_batch_number) ——對應到 Enhancement Spot ES_BATCH_NUMBER_INT、BAdI 定義 BADI_BATCH_NUMBER_INT (套件 VB, singleUse="true", Interface IF_EX_BADI_BATCH_NUMBER_INT)
2. Classic User-Exit (CALL CUSTOMER-FUNCTION '001' / '002') ——這就是本題要用的：EXIT_SAPLV01Z_001 (Determination of Internal Number Ranges for Automatic Batch Number) 與 EXIT_SAPLV01Z_002 (Modification of New Batch Numbers from Internal Assignment)，兩者都掛在 SAP Enhancement SAPLV01Z (說明：「CFCs for internal batch number assignment」, Function Group XVBZ, 套件 VBEX) 底下
3. S/4HANA Cloud BAdI (g_badi_batch_number_cust, IF_LOBM_BATCH_NUMBER) ——最新一代的 Key User Extensibility 機制

三者的參數幾乎一模一樣 (NR_RANGE_NR / OBJECT 預設 'BATCH_CLT' / SUBOBJECT / TOYEAR)，這證實了 en01 提到的現象不是特例：SAP 在同一個擴充點上，把新舊技術疊在一起，不會因為有了新技術就砍掉舊的，舊系統用 Classic User-Exit 寫的客製化，升版後照樣有效。

兩支 Function Exit 分工：

- EXIT_SAPLV01Z_001 (Include ZXVBZU01)：在取號之前執行，CHANGING 參數可以把 OBJECT (預設 'BATCH_CLT')、NR_RANGE_NR (預設 '01') 換成自己的 Number Range Object——決定「跟哪個號碼簿要號碼」
- EXIT_SAPLV01Z_002 (Include ZXVBZU02)：在取號之後執行，CHANGING NEW_CHARG 拿到剛取到的號碼、可以重新加工——決定「拿到號碼之後怎麼組成最終字串」

本題只用 EXIT_SAPLV01Z_002：不去重導向 OBJECT (EXIT_SAPLV01Z_001 用的 Include ZXVBZU01 在這套系統裡還沒被生成過，且 ZX 開頭的 Include 有保留規則，無法用一般 ADT Include 建立 API 生出來，見下方「查

證與踩坑記錄」)，改成直接在 `_002` 裡自己呼叫 `NUMBER_GET_NEXT` 對一個全新的、跟標準 `BATCH_CLT` 完全獨立的 **Number Range Object** 取號，取到號碼後加工成最終格式、蓋掉 `NEW_CHARG`——效果相同，但完全不用碰 `_001`。

⚠ **更正**：寫程式碼進 **Include** 並啟用，還不等於這個 **Enhancement** 真的生效——這點原本（第一版教材）誤判了。Classic User-Exit 要真的運作，一定要走 **CMOD** 完整跑完：① 建一個 **Enhancement Project**（Project 名稱自訂）② 在 Project 裡 Assign Enhancement **SAPLV01Z**（畫面上會列出這個 Enhancement 底下的所有 Component，含 `EXIT_SAPLV01Z_001/_002`）③ 雙擊 `EXIT_SAPLV01Z_002` 進去編輯（畫面會帶你到 **ZXVBZU02** 這個 Include，內容應該已經是我們寫好的程式碼）④ **Activate Project**——這一步才是真正的開關，沒做這步，就算 Include 原始碼已經在系統裡、也用 ADT 啟用過，Function Group 實際載入執行的版本還是不會反映這段程式碼。這是 GUI-only 步驟，**CMOD** 沒有 ADT API（見 `.claude/rules/sap-adt-mcp.md` 第 20 節），由使用者在 SAP GUI 操作，Claude 只能把 Include 內容準備好、驗證語法沒問題。

Number Range Object (NROB) 概念：一個「號碼簿」，本身有名稱（如 `BATCH_CLT`）、一或多個「Interval（區間）」（用二字节代碼區分，如 `'01'`），每個 Interval 記錄 From / To / 目前用到哪裡（**NRLEVEL**）。ABAP 呼叫 `NUMBER_GET_NEXT`（`EXPORTING nr_range_nr / object / IMPORTING number`）就能拿到「這個 Interval 目前的下一號」，系統自動處理並發（同時兩個人呼叫不會拿到同一號）。⚠ **這個工具本身完全沒有 ADT API**（跟 SM59/DBCO/SPAD/Search Help 同一類 GUI-only），只能在 **SNRO** 交易碼手動建立與維護 Interval；`NUMBER_GET_NEXT` 回傳的 `number` 一律是 **10 碼、左邊補零的數字字串**，不管 Interval 本身定義的號碼位數是多少（本題已用真實執行結果驗證：`0000000001 / 0000000002 / 0000000003`）。

學習目標

- 能講出 Classic User-Exit（`EXIT_SAPLV01Z_001/_002`）、新式 BAdI（`BADI_BATCH_NUMBER_INT`）、Cloud BAdI 三代技術在同一個標準流程裡並存的現象，理解「新技術不會取代舊技術」
- 能分辨 `_001`（取號前，決定用哪個 Number Range Object）與 `_002`（取號後，加工最終字串）兩個插入點的職責差異
- 能講出 Number Range Object 的核心概念（Object / Interval / NRLEVEL），知道要用 **SNRO** 手動建立，`NUMBER_GET_NEXT` 回傳值一律 10 碼補零
- 能講出批號欄位 **CHAR(10)** 的長度限制如何反過來限制了「日期+時間+流水號」格式的設計取舍
- 知道 **ZX** 開頭的 Include 是保留給 Exit Function Group 的特殊命名空間，一般 ADT Include 建立 API 建不出來

事前準備（已於本系統 client 130 實際完成，非假設）

- Number Range Object ZEN02BAT**：使用者於 **SNRO** 手動建立，Number Length Domain 填 **CHARG**（查證自標準物件 `BATCH_CLT` 在設定表 **TNRO** 的 **DOMLEN** 欄位，同一個 Domain），Interval **01**：`0000000001 ~ 9999999999`，未勾 External。已用資料預覽 API 查 **TNRO** / **NRIV** 兩張表確認設定正確。
- EXIT_SAPLV01Z_002** 的 Include **ZXVBZU02**：查證時發現這個 Include 已經是一個真實物件（套件 **ZPP**，2023-08-08 由另一位同事 **MAVIS** 建立，但內容是空的）；進一步查 **MODSAP** / **MODACT** 兩張表確認 Enhancement **SAPLV01Z** 從未被任何 **CMOD Project** 正式指派，SE10 也查無掛著這支程式的傳輸請求——判斷是當年被雙擊觸發自動產生、但沒人接著建 Project 或寫程式碼的殘留物件。**確認同事無異議後**，用傳輸請求 **S4HK901982**（套件 **ZPP**）寫入並啟用（Include 原始碼層級）。⚠ **這一步只完成了原始碼，Enhancement 尚未真的生效**——還需要使用者在 **CMOD** 建 Enhancement Project、Assign **SAPLV01Z**、Activate Project 才會真的啟用（見 Lecture 的更正說明），這部分由使用者在 GUI 處理，Claude 沒有 ADT API 可以做這一步。

3. 驗證程式 `ZR_EN02_BATCH_DEMO ( $TMP )`：不動任何真實貨物移動，單獨呼叫三次 `NUMBER_GET_NEXT` 驗證 `ZEN02BAT` + 格式化邏輯，已用 `programrun` 無頭執行，真實輸出：

```
`` raw number: 0000000001 batch no: 2607280001 raw number: 0000000002 batch no:
2607280002 raw number: 0000000003 batch no: 2607280003 ``
```

(執行當下系統日期 2026-07-28，`sy-datum+2(6)` = 260728，補上 4 碼流水號 = 10 碼批號)

題目需求

- 1. 完成三代擴充技術對照表：技術世代 / 呼叫語法 / 對應物件名稱 / 是否需要 `CMOD Project`。
- 2. 解釋為什麼本題選擇只實作 `EXIT_SAPLV01Z_002`，不實作 `_001`：說明 `ZXVBZU01` 目前的狀態，以及 `ZX` 開頭 `Include` 的建立限制。
- 3. 讀懂 `ZXVBZU02` (見答案快照 `zxvbzu02.prog.abap`)，指出：`NEW_CHARG` 這個變數為什麼不用自己宣告 (提示：它是 `EXIT_SAPLV01Z_002` 的 `CHANGING` 參數，`Include` 是直接嵌進 `FUNCTION` 內部，參數在整個 `Include` 範圍內都可以直接當變數用)。
- 4. 情境判斷：如果之後想要「依工廠分開計數」(不同工廠各自一組流水號，互不影響)，`Number Range Object` 要怎麼調整？(提示：見 `Lecture` 提到的 `Subobject` 概念)

參考答案

三代擴充技術對照表：

技術世代	呼叫語法	對應物件	需要 CMOD Project ?
Classic User-Exit	<code>CALL CUSTOMER-FUNCTION '001' / '002'</code>	<code>EXIT_SAPLV01Z_001/_002</code> (Enhancement <code>SAPLV01Z</code>)	是，缺一不可——一定要用 <code>CMOD</code> 建 Enhancement Project、Assign <code>SAPLV01Z</code> 、Activate Project，這個 Enhancement 才會真的生效；Include 原始碼寫好、用 ADT 啟用過，只代表原始碼存在系統裡，Function Group 實際載入執行的版本不會反映，直到 Project 被 Activate (第一版教材誤判成「不需要 Project」，已更正)
Classic BAdI (新式，Enhancement Spot 管理)	<code>GET BADI / CALL BADI</code>	<code>BADI_BATCH_NUMBER_INT</code> (Enhancement Spot <code>ES_BATCH_NUMBER_INT</code>)	否，直接 SE18/SE19 建 Implementation

技術世代	呼叫語法	對應物件	需要 CMOD Project ?
S/4HANA Cloud BAdI	透過 g_badi_batch_number_cust (IF_LOBM_BATCH_NUMBER)	Key User Extensibility BAdI	否，走 ABAP Cloud 擴充性框架

為什麼只做 **_002** : **_001** 的 Include **ZXVBZU01** 在這套系統從未被生成過，且已實測確認 **ZX** 開頭的 Include 名稱保留給 **Exit Function Group** 專用，直接用標準的 ADT Include 建立 API (POST /sap/bc/adt/programs/includes) 會被拒絕 (Program names ZX... are reserved for includes of exit function groups)，必須先在 SE37/CMOD 用 GUI 觸發生成才能寫入——這超出本題範圍。改成完全在 **_002** (Include 已存在) 裡自己呼叫 **NUMBER_GET_NEXT** 指定 **ZEN02BAT**，不依賴 **_001** 重導向 **OBJECT** 參數，效果一樣，且不需要碰觸尚不存在的 **_001**。

情境判斷 (依工廠分開計數) : Number Range Object 要重新設計成帶 **Subobject** (在 **SNRO** 建立時填 **Subobject Data Element**，例如工廠代碼 **WERKS_D**)，每個工廠各自維護一組 **Interval**；呼叫 **NUMBER_GET_NEXT** 時要多帶 **subobject** 參數 (傳入實際工廠代碼)，系統會依 **OBJECT+SUBOBJECT** 的組合各自計數，不會互相影響。

思考題

- 三代技術裡，**_001** / **_002** 這種 Classic User-Exit 沒有 BAdI 那種 **TRY...CATCH cx_badi_not_implemented** 的保護機制，這代表什麼實務風險？ (提示：如果 Include 裡的程式碼寫錯、丟出未攔截的例外，會直接讓整個 **VB_NEXT_BATCH_NUMBER** 呼叫中斷，不像 BAdI 沒實作時會被優雅地跳過——Classic User-Exit 的程式碼品質要求其實更高，因為沒有安全網)
- ZXVBZU02** 這個 Include 已經是套件 **ZPP** 的物件，但從未被任何 CMOD Project 指派過——如果之後 PP 團隊突然想要用 **EXIT_SAPLV01Z_002** 做他們自己的事，會發生什麼衝突？ (提示：一個 Function Exit 的 Include 只有一份，兩個團隊的需求會被迫寫在同一支程式碼裡，這正是為什麼動手前跟同事確認這一步在真實企業環境中不能省略——不像新建 Z 物件那樣互不干擾)
- 為什麼 **NUMBER_GET_NEXT** 的 **number** 輸出參數固定是 10 碼補零，而不是依 Number Range Object 定義的區間位數決定長度？這對本題「取後 4 碼當流水號」的寫法有什麼啟示？ (提示：不管 Interval 定義多少位數，程式都要自己決定「取哪一段」，**+6(4)** 這種寫法要跟 Interval 實際的最大值位數對齊，如果 Interval 改成 5 位數上限就要跟著改成取後 5 碼，否則流水號可能被截斷或補零位置算錯)

答案

ZEN02BAT (Number Range Object，使用者於 **SNRO** 建立，Domain **CHARG**，Interval **01** : **0000000001 ~ 9999999999**)、**ZXVBZU02** (**EXIT_SAPLV01Z_002** 客戶 Include，套件 **ZPP**，傳輸請求 **S4HK901982**，快照 **zxvbzu02.prog.abap**)、**ZR_EN02_BATCH_DEMO** (驗證程式，**\$TMP**，快照 **zr_en02_batch_demo.prog.abap**) 均已建立並啟用 (**sap_inactive_objects** 確認無殘留未啟用版本)。已用 **programrun** 無頭執行驗證程式 (不經過 Enhancement 機制，直接呼叫 **NUMBER_GET_NEXT**)，真實輸出批號 **2607280001 / 2607280002 / 2607280003**，證實 **ZEN02BAT** + 格式化邏輯本身正確。三代擴充技術對照表、Classic User-Exit 選用理由、情境判斷見本題內文。

⚠ 待辦 (**GUI-only**，由使用者處理) : **EXIT_SAPLV01Z_002** 這個 Enhancement 本身尚未真的啟用——Include 原始碼已就緒，但還需要在 **CMOD** 完整跑一次「建 Enhancement Project → Assign **SAPLV01Z** → Activate Project」，才會讓 **MIGO** 等真實貨物移動的批號自動給號真的套用這段新邏輯。在使用者完成 **CMOD**

這一步之前，若照前一輪對話提到的方式用真實採購單做 Goods Receipt 測試，批號應該還是舊的預設格式，不代表程式邏輯有問題。